

INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
KALANKI, KATHMANDU
(AFFILIATED TO TRIBHUVAN UNIVERSITY)



PROJECT REPORT
ON
BANK MANAGEMENT SYSTEM

SUBMITTED BY:
Aman Kumar Ray (ACE080BCT010)

SUBMITTED TO:
Department Of Electronics and Computer Engineering.

SUBMISSION DATE: 2082-11-15

INTRODUCTION

A Bank Management System in SQL is a database-driven system that uses Structured Query Language (SQL) to store, manage, and manipulate banking data efficiently. It is designed using a Relational Database Management System (RDBMS) such as MySQL or Microsoft SQL Server.

In this system, all banking information is stored in the form of tables with rows and columns. These tables are related to each other using primary keys and foreign keys to maintain data integrity.

DOMAIN: BANK MANAGEMENT SYSTEM

The system manages:

- Customers
- Bank Accounts
- Bank Transactions
- And Loan

ENTITY LIST

1. Customer
2. Account
3. Transaction
4. Loan

Schema with Data Types

Customer Table

COLUMN	DATATYPE
customer_id	Int, primary key
name	Varchar (50)
phone	Varchar (15)
address	Varchar (100)

Account Table

COLUMN	DATATYPE
account_id	Int, primary key
customer_id	Int, foreign key
account_type	Varchar (20)
balance	DECIMAL(10,2)

BankTransaction Table

COLUMN	DATATYPE
transaction_id	Int, primary key
account_id	Int, foreign key
transaction_type	Varchar (20)
amount	DECIMAL(10,2)
transaction_date	DATE

Loan Table

COLUMN	DATATYPE
loan_id	Int, primary key

customer_id	Int, foreign key
loan_amount	DECIMAL(10,2)
loan_type	Varchar (30)

SQL QUERIES IMPLEMENTATION ANALYSIS

JOIN OPERATIONS

In our Bank Management System, we used JOIN operations to combine data from multiple related tables.

Since data is stored in separate tables (Customer, Account, Loan, BankTransaction), JOIN helps us retrieve meaningful combined information.

INNER JOIN

- Returns only matching records from both tables.
- Example used: Customer + Account
- Purpose: To display customer names with account details.
- It shows only customers who have accounts.

LEFT JOIN

- Returns all records from the left table and matching records from the right table.
- Example used: Customer + Loan
- Purpose: To show all customers, even if they don't have a loan.

Why we used JOIN:

- To maintain normalization (avoid data redundancy).
- To fetch relational data efficiently.
- To demonstrate the relationship between entities.

AGGREGATE FUNCTIONS

Aggregate functions like (count, sum, avg, group by) perform calculations on multiple rows and return a single result. It is used to perform statistical and financial analysis. It also summarizes large data into meaningful results. In this project we use aggregate function:

- To calculate the total balance per account type.
- To calculate the average loan amount.
- Groups rows having the same values in a column.
- To summarize large data into meaningful results.

SUBQUERY

A subquery is a query inside another query. In this project we use subquery to:

- Find customers whose loan amount is greater than the average loan.
- Find accounts with a balance greater than average balance.

It performs advanced filtering, compares values with calculated results. Show nested query concepts.

VIEW

A View is a virtual table created from a SELECT query, we created CustomerAccountView. The main purpose to create view:

- To simplify complex queries.
- To improve security (hide original table structure).
- Does not store data physically.

TRANSACTION (COMMIT & ROLLBACK)

Transactions ensure data consistency and integrity. A transaction is a sequence of SQL operations treated as a single unit.

COMMIT

- Saves all changes permanently.
- Used when a transaction is successful.

ROLLBACK

- Cancels all changes made during transaction.
- Used when error occurs.

Example used:

- Deducting money from account balance.

Why Transactions are Important:

- Prevent data inconsistency.
- Ensure ACID properties:
 - Atomicity
 - Consistency
 - Isolation
 - Durability

SOURCE CODE

Below is the source code link attached for Bank Management System project:

https://github.com/amann45/Bank_Management_Project_SQL/blob/main/bank_management_system.sql

Repository link: https://github.com/amann45/Bank_Management_Project_SQL

CONCLUSION

The Bank Management System project was designed to demonstrate the practical implementation of core Database Management System (DBMS) concepts. The system includes four main entities: Customer, Account, Transaction, and Loan. These entities are connected using primary and foreign key relationships to maintain referential integrity and proper relational structure. Various SQL concepts (JOIN Operations, Aggregate functions, Subquery, View, Transaction) were implemented to maintain data integrity and demonstrate ACID properties.

Hence, we concluded that the Bank Management System project successfully fulfills all the required DBMS concepts and objectives. It demonstrates how structured database design helps in managing real-world banking operations efficiently.